HT_MK1 Harness Tester

Code Walkthrough — Architecture and Call Hierarchy

Document ID:	HT_MK1_SDD
Document Version:	1.0
Date:	26 August 2026
System:	Control Card + Matrix Card + HV Card + Operator GUI
Prepared by:	Azista BST Aerospace Private Limited
Prepared for:	Infinity Integration Pvt. Ltd.
Classification:	Confidential

Revision History

Version	Date	Description	Author
1.0	26 August 2026	Initial release	[Author]

Table of Contents

1. Introduction

1.1 Purpose

This document explains how the HT_MK1 firmware and GUI source code are structured: which files call which functions, in what order, and what each function does. It is written for an engineer who will read, maintain, or extend the code, using the delivered source alongside this document.

1.2 Scope

Covers the embedded firmware (Core/Inc, Core/Src) and the GUI application (gui_flutter/lib). For hardware design, refer to the project schematics. For operating the system, refer to the User Manual. For programming a unit, refer to the Flashing Procedure. For acceptance testing a built unit, refer to the Functional Test Procedure.

1.3 How to Read the Call-Hierarchy Trees

Sections 3 and 4 include call trees in this form:

```
caller()
├── firstCallee()
│   └── nestedCallee()
└── secondCallee()
```

Each line is a function call made by the function above it, indented one level per call depth.

Every function name in a tree or table is exactly as it appears in the delivered source and can be searched for directly.

1.4 Definitions and Abbreviations

Term	Meaning
MCU	Microcontroller Unit — the single processor chip that runs the firmware.
RTOS	Real-Time Operating System. This project uses FreeRTOS: several software threads run on the one MCU.
I2C, SPI	Digital communication buses used between chips on the boards.
ADC / DAC	Analog-to-Digital / Digital-to-Analog Converter.
IDAC	Current-source output of the resistance-measurement ADC; pushes a known test current through a wire.
PGA	Programmable Gain Amplifier — sets how much a small signal is amplified before it is measured.
Netlist	The list of expected wire connections for the harness under test.
Fixture	Which physical connector the harness is currently on: the Matrix connector or the HV connector.

1.5 References

- Project schematics: Control Card, Matrix Card, HV Card.
- HT_MK1 User Manual, Flashing Procedure, Functional Test Procedure.

2. System Overview

The HT_MK1 tests up to 256 wires in a harness in three stages — continuity, 4-wire (Kelvin) resistance, and 500 V insulation. The software is two programs that talk over a serial link:

Component	Runs on	Role
Firmware	One STM32G474 MCU, on the Control Card	Owns all hardware. Runs every test. The Matrix Card and up to four HV Cards have no processor of their own; the firmware drives them over I2C/SPI.
GUI	Windows desktop (Flutter)	Sends commands, receives results, manages netlists. No test logic of its own — it is a client of the firmware's serial protocol (Section 5).

3. Firmware — Architecture and Call Hierarchy

3.1 File Layout

Folder	Files	Layer
drivers/	mcp23017, ads124s08, ad7476, dac8830, ds18b20, ads1232 (bench-only, compiled out by default)	Chip-level. Talks to hardware registers over I2C/SPI/GPIO. No test logic.
cards/	matrix_card, hv_card, control_frontend	Board-level. Combines drivers into card operations (route a pin, close a relay, set a mode).
test/	continuity, kelvin, insulation	Test-level. One algorithm per file, built from card-level calls.
bsp/board	board	Start-up. Instantiates and initializes every driver/card instance used by the layers above.
app/	tasks, proto, log	Application-level. RTOS threads, the command queue, the serial protocol, logging.

Each layer only calls into the layer(s) below it — app/ calls test/ and cards/, test/ calls cards/ and drivers/, cards/ calls drivers/. Nothing in drivers/ or cards/ calls upward into test/ or app/.

3.2 Startup Call Hierarchy

```
main()                                              Core/Src/main.c
├── HAL_Init(), SystemClock_Config()
├── MX_I2C2_Init(), MX_I2C3_Init(), MX_SPI1_Init(), MX_SPI2_Init(), MX_SPI3_Init(), ...
│                                                    (CubeMX-generated peripheral setup)
├── Board_Init()                                    bsp/board.c
│   ├── board_init_matrix()
│   ├── board_init_ads124s08()
│   ├── board_init_frontend()
│   ├── board_init_hv(i)                            once per fitted HV card (BOARD_HV_COUNT)
│   └── board_init_temp()                           failure here is non-fatal
├── osKernelInitialize()
├── MX_FREERTOS_Init()
```

```
│   └── Tasks_Init()                              app/tasks.c
│       ├── Log_HwInit_LPUART1(), Log_Init()
│       ├── osMutexNew()          -> s_hwmtx      (the one hardware-access mutex)
│       ├── osMessageQueueNew()   -> s_cmdq, s_rxq
│       └── osThreadNew() x4      -> tLogger, tSafety, tSequencer, tComms
└── osKernelStart()                               (the four threads above begin running)
```

Core/Src/main.c, Core/Src/app_freertos.c, Core/Src/bsp/board.c, Core/Src/app/tasks.c.

Board_Init() aborts the whole chain on the first failure, except board_init_temp() — a missing temperature sensor must not stop the electrical-test hardware from coming up. Board_IsReady() records the outcome; proto.c checks it before any hardware command.

3.3 Runtime Threads

Thread	Priority	Entry function	Role
tSafety	High	SafetyTask()	Always-on loop. Forces every card to a safe state on a latched fault, and backstops HV-off whenever no test is running.
tComms	AboveNormal	CommsTask()	Reads bytes from the UART queue and feeds the protocol parser. Kept above tSequencer so a command — especially ABORT — is always received even mid-test.
tSequencer	Normal	SequencerTask()	Blocks on the command queue; runs one command at a time, holding the hardware mutex for its duration.
tLogger	Low	Log_Task()	Drains the debug log queue to the UART.

3.4 Function Reference — app/ (tasks.c, proto.c)

Function	Calls	Purpose
Tasks_Init()	Log_HwInit_LPUART1(), osMutexNew(), osMessageQueueNew() x2, osThreadNew() x4	One-time boot setup: logger, hardware mutex, command/rx queues, the four RTOS threads.
SafetyTask()	force_safe_all(), HvCard_HvEnable()	Runs forever. See the tree in Section 3.2 caption and the snippet below.
SequencerTask()	run_command()	Runs forever. Blocks on s_cmdq; hands each dequeued command to run_command().
run_command()	Continuity_TestPair(), Kelvin_MeasurePair(), Insulation_TestPair(), run_continuity_all(), run_resistance_all(), run_insulation_all(), Board_ScanBus(),	Dispatches one dequeued command by type (switch statement); acquires s_hwmtx first.

	DS18B20_ReadTemperature(), force_safe_all()	
run_resistance_all()	Kelvin_MeasurePair(), Proto_EvtRes(), Proto_EvtProgress(), Proto_EvtDone()	Loops the loaded netlist; one Kelvin measurement per pair; streams results as they complete.
run_continuity_all()	Continuity_TestPair(), Proto_EvtCont(), Proto_EvtProgress(), Proto_EvtDone()	Same shape as run_resistance_all(), for continuity (verify mode, or full discovery scan).
run_insulation_all()	Insulation_TestPair(), Proto_EvtInsul(), Proto_EvtProgress(), Proto_EvtDone()	Same shape again, for insulation. Requires INSUL ARM to have been accepted first.
CommsTask()	Proto_RxByte()	Runs forever. Blocks on the byte queue fed by the UART receive interrupt.
Proto_RxByte()	proto_exec()	Assembles bytes into a line; calls proto_exec() once a full line (ending '\n') is buffered.
proto_exec()	proto_split(), proto_hw_ready(), proto_post(), proto_post_run()	Parses one command line and dispatches it by string comparison. See Section 3.6.
proto_post_run()	proto_post()	Refuses with ERR EBUSY if a run is already active; otherwise clears the abort flag and posts.
proto_post()	Tasks_PostCommand()	Queues one command; replies <OK started, or ERR EBUSY if the queue is full.

3.5 Function Reference — test/ and cards/

Function	Calls	Purpose
Continuity_TestPair()	MatrixCard_ConnectPair(), Frontend_SetMode(), AD7476_ReadVolts()	Routes one wire pair, reads a voltage, judges open/connected/anomaly by voltage band.
Kelvin_MeasurePair()	MatrixCard_SetSensePaired(), MatrixCard_ConnectPair(), Frontend_SetMode(), ADS124S08_SetIdac(), ADS124S08_SetMux(), ADS124S08_SetGain(), ADS124S08_BypassPga(), ADS124S08_ConvertOnce(), ADS124S08_OhmsRatiometric(), ADS124S08_OhmsFromCurrent()	Full 4-wire, auto-ranged, ratiometric resistance measurement. See Section 3.7 for the exact call order.
Insulation_TestPair()	HvCard_ConnectPair(), HvCard_SetVoltageFraction(), AD7476_ReadVolts() (via HvCard_ReadLeakageVolts()), HvCard_HvOff()	Cold-switches relays, applies HV, reads the leakage node, always de-energizes and opens relays on exit.
MatrixCard_ConnectPair()	MCP23017_WritePin()	Closes the mux channels routing one HI and one LO pin to the shared force/sense buses.
HvCard_ConnectPair()	MCP23017_WritePin()	Closes one inject relay and one return relay on the addressed HV card.
Frontend_SetMode()	HAL_GPIO_WritePin() (OPT0_CNTR)	Switches the shared analog front end between continuity-divider and impedance mode.

3.6 Runtime Call Hierarchy — RES RUN

Command arrival (tComms) and command execution (tSequencer) are two separate call chains, joined by the command queue:

```
CommsTask()                                          app/tasks.c
└── Proto_RxByte()            [once per received byte]     app/proto.c
    └── proto_exec("RES RUN")                 [called on the terminating '\n']
        ├── proto_hw_ready()  -> Board_IsReady()
        └── proto_post_run(CMD_RES_RUN)
            └── proto_post()
                └── Tasks_PostCommand()  -> osMessageQueuePut(s_cmdq)
                                            reply sent: <OK started

SequencerTask()                                      app/tasks.c
└── run_command()             [dequeues the command; holds s_hwmtx for the whole run]
    └── run_resistance_all()
        └── Kelvin_MeasurePair()      [once per netlist pair]      test/kelvin.c
            ├── MatrixCard_SetSensePaired() / MatrixCard_ConnectPair()   cards/matrix_card.c
            ├── Frontend_SetMode(FRONTEND_MODE_IMPEDANCE)                cards/control_frontend.c
            ├── ADS124S08_SetIdac(2 mA) -> settle -> ConvertOnce()       (excited reading, DUT)
            ├── ADS124S08_SetMux(AIN8) / BypassPga() / ConvertOnce()     (excited reading,
reference R131)
            ├── ADS124S08_SetIdac(OFF) -> settle -> ConvertOnce() x2     (zero-current baseline,
both channels)
            └── ADS124S08_OhmsRatiometric()   or   ADS124S08_OhmsFromCurrent()  (fallback)
        Proto_EvtRes() + Proto_EvtProgress()         [sent after every pair]
        ... repeats for every pair in the netlist, checking Proto_AbortRequested() each time ...
        Proto_EvtState("idle") -> Proto_EvtDone("res", pass, fail)
```

Verified against Core/Src/app/tasks.c (run_command(), run_resistance_all()), Core/Src/app/proto.c (proto_exec(), proto_post_run(), proto_post()), and Core/Src/test/kelvin.c (Kelvin_MeasurePair()).

CONT RUN and INSUL RUN follow the identical two-chain shape — proto_exec() posts to the queue, SequencerTask() dequeues and calls run_continuity_all() / run_insulation_all() — substituting Continuity_TestPair() / Insulation_TestPair() for Kelvin_MeasurePair() inside the loop. INSUL RUN additionally requires INSUL ARM to have set the armed flag first, checked before the loop starts.

ABORT does not go through the command queue at all — proto_exec() sets a flag inline and replies immediately, because tSequencer holds s_hwmtx for the whole run and a queued command could not preempt it. The run loops (run_resistance_all(), etc.) poll this flag after every pair.

3.7 Notable Implementation Details

SafetyTask() — the entire independent safety backstop, unconditional on what the sequencer is doing:

```
static void SafetyTask(void *arg)
{
  for (;;)
  {
    if (s_fault)
```

```
    {
      if (osMutexAcquire(s_hwmtx, 50U) == osOK)
      {
        force_safe_all();
        osMutexRelease(s_hwmtx);
      }
    }
    else if (!s_hv_active)
    {
      for (i = 0; i < BOARD_HV_COUNT; i++)
        HvCard_HvEnable(&g_hv[i], 0U);   /* backstop: HV off whenever idle */
    }
    osDelay(10U);
  }
}
```

tasks.c, SafetyTask() (from line 152).

`Kelvin_MeasurePair()` — the ratiometric calculation this test exists to make possible:

```
if (ref_ok && (ref_excited - ref_zero) != 0)
{
  res->resistance_ohm = ADS124S08_OhmsRatiometric(
      res->code, gain_dut, ref_excited - ref_zero, ADS124S08_GAIN_1,
      KELVIN_CAL_R_REF_OHM);
  res->ratiometric = 1U;
}
else
{
  res->resistance_ohm = ADS124S08_OhmsFromCurrent(&g_ads124s08, res->code,
                                                  KELVIN_FORCE_CURRENT_A);
}
```

kelvin.c, Kelvin_MeasurePair() (from line 190).

4. GUI — Architecture and Call Hierarchy

4.1 File Layout

Folder	Files	Layer
htproto/	codec.dart, messages.dart, connection.dart, serial_transport.dart, simulator.dart, netlist_file.dart	Wire protocol only. Encodes commands, parses replies/events, manages the connection. No UI code.
design/	tokens.dart, widgets.dart, painters.dart, model.dart	Presentation primitives — colors, reusable widgets, canvas painters (wiring diagram, histogram).
app/	app_state.dart, shell.dart, parts.dart, modals.dart, report.dart, run_history.dart, build_tag.dart	Application state and cross-screen chrome (status bar, nav rail).
views/	run_view.dart, cont_view.dart, res_hv_views.dart, misc_views.dart	The seven screens. Each is a thin layer that reads AppState and calls its methods — no protocol code here.

4.2 Startup Call Hierarchy

```
main()                                              lib/main.dart
├── parse launch options (--serial / --sim / --host / --port / --scenario / --nets)
├── AppState(...)            [constructed once; holds every screen's state]
├── state.refreshPorts()
└── runApp(HtApp(state: state, ...))
    └── HtHome  (StatefulWidget)                     lib/main.dart
        └── AnimatedBuilder(animation: state, ...)   [rebuilds on every state.notifyListeners()]
            └── AnimatedSwitcher(child: <the view named by state.view>)
```

lib/main.dart. There is no router — the visible screen is just a string field on AppState.

4.3 Function/Method Reference

File : Function/Method	Calls	Purpose
app_state.dart : runCont() / runRes() / runHv()	_startRun(), _cmd()	Per-test entry points, called directly from each view's Run button.
app_state.dart : _startRun()	_cmd(), _beginRun()	Sends the command; only marks the run "started" (running=kind) after the instrument's reply is accepted.
app_state.dart : _cmd()	ConnectionManager.execute()	Thin wrapper: calls execute() and reports a refusal if the reply is an error.
app_state.dart : onEvent()	_onCont(), _onRes(), _onInsul(), _onProgress(), _onDone(), notifyListeners()	Dispatches every incoming ! event by its runtime type. See Section 4.5.
connection.dart : execute()	_execute()	Public entry point; returns the first (and normally only) reply message.
connection.dart : _execute()	_sendLine()	Registers a pending-reply entry, sends the command, awaits it with a timeout Timer.
connection.dart : _sendLine()	Transport.send()	Writes the command bytes to the transport; logs and mirrors them to the wire log.
connection.dart : _handleIncoming()	parseLine(), onEvent callback	Called once per complete received line; routes it as a reply match or an event.
codec.dart : parseLine()	_parseReply(), _parseEvent()	Dispatches on the leading byte ('<' or '!') and builds a typed Message subclass.
netlist_file.dart : _guessFixture()	_guessShape()	Infers connector layout (DB9/DB37/circular/rect/etc.) from a netlist's Conn ID / Part Number columns.

4.4 Runtime Call Hierarchy — Button Press to Wire

```
ContView "Run continuity" button                    views/cont_view.dart
└── AppState.runCont()                                app/app_state.dart
    └── _startRun('cont', commands.contRun(mode), ...)
        └── _cmd(wire)
            └── ConnectionManager.execute(wire)        htproto/connection.dart
```

```
                └── _execute(wire)
                    ├── _sendLine(wire) -> Transport.send()   [writes bytes, logs tx]
                    └── await entry.completer.future          [resolves on the matching '<'
reply, 2 s timeout]
    (only once the reply resolves)
    -> _beginRun() sets running = 'cont' -> notifyListeners() -> screen redraws
```

Verified against gui_flutter/lib/app/app_state.dart and lib/htproto/connection.dart.

4.5 Runtime Call Hierarchy — Incoming Line to Screen Update

```
Transport.incoming  (Stream<Uint8List>)             htproto/serial_transport.dart or
simulator.dart
└── LineFramer.feed()                                 htproto/connection.dart
    └── _handleIncoming(line)        [once per complete '\n'-terminated line]
        ├── parseLine(line)                           htproto/codec.dart
        │   └── returns a typed Message  ('<' -> reply, '!' -> event)
        ├── if reply: matched to the oldest pending command -> completes its Future
        └── if event: AppState.onEvent(m)              app/app_state.dart
            └── switch (m)
                ├── ContResult()  -> _onCont(m)
                ├── ResResult()   -> _onRes(m)
                ├── InsulResult() -> _onInsul(m)
                ├── SafeEvent()   -> handling = 'safe'   (the ONLY place this is set true)
                └── ... one case per event type ...
                    -> notifyListeners() -> AnimatedBuilder rebuilds the visible screen
```

Verified against gui_flutter/lib/htproto/connection.dart (line 443) and lib/app/app_state.dart (line 1905).

4.6 Notable Implementation Details

`codec.dart : commands.contRun()` — how a typed call becomes an exact wire command:

```
static Uint8List contRun(ContMode mode) =>
    _line('CONT RUN ${mode.wire}');   // writes:  >CONT RUN verify\n
```

codec.dart, commands.contRun() (line 268).

Netlist connector-ID handling (netlist_file.dart): a netlist file can reuse the same connector ID for two different roles — its own mated pair, and a reference from a different connector's row. Treating both as one combined pin range let a foreign reference silently widen a connector's guessed size. The fix tracks native and foreign observations separately:

```
class _ConnObs {
  int? nativeMin, nativeMax;         // this connector's own mated-pair rows
  int? foreignSrcMin, foreignSrcMax; // OTHER connectors' rows that merely
  int? foreignDstMin, foreignDstMax; // reference this connector's pins
}
// A connector's guessed range is built from nativeMin/nativeMax whenever
// a native block exists at all - foreign observations never widen it.
```

netlist_file.dart, _ConnObs (line 347) and the range logic that follows it.

5. Firmware-GUI Interface (Serial Protocol)

Plain text lines over a serial connection. Every line starts with a marker:

Marker	Meaning
>	A command sent from the GUI to the firmware.
<	A reply from the firmware, immediately after a command. Exactly one reply per command.
!	An event sent by the firmware on its own — a result, a progress update, a state change.
#	A human-readable log line, for display only.

5.1 Commands

Command	Firmware handler	What it does
PING / ID / STATUS	proto_exec()	Basic health check and version/state query.
SAFE	proto_exec() -> CMD_FORCE_SAFE	Forces HV off, all relays open.
ABORT	proto_exec() (inline, not queued)	Stops the current run immediately.
NETLIST BEGIN/ADD/END/GET	proto_exec()	Loads the expected-wiring list, up to 256 pairs.
CONT RUN	run_continuity_all()	Continuity over the netlist, or full discovery scan.
RES RUN	run_resistance_all()	Resistance over the netlist. See Section 3.6.
INSUL ARM / INSUL RUN	run_insulation_all()	Arm, then run, the insulation test.
LIMITS GET / SET	proto_exec()	Reads/sets the resistance and insulation pass/fail limits.
FAULT CLEAR	proto_exec() -> CMD_CLEAR_FAULT	Clears a latched fault.
MANUAL PATH / MANUAL SWEEP	run_manual_sweep(), CMD_CONTINUITY	Engineering: test one pair, or one pin against all others.
MANUAL RELAY	proto_exec() (always refused)	Always ERR EHW — driving one HV relay by hand is not permitted.
FIXTURE	Proto_SetFixture()	Tells the firmware which connector the harness is on.

5.2 Events and Error Codes

Event	Meaning
!PROGRESS	How far through the current run.
!CONT / !RES / !INSUL	One result, sent as soon as it is available.
!DONE	The run finished; pass/fail summary.
!STATE	Overall state; also a periodic heartbeat when idle.
!SAFE	The only signal the GUI treats as "safe to handle".
!FAULT	A fault occurred, with a code and text.

Code	Meaning
EBUSY	A run is already in progress, or the queue is full.
EFIXTURE	Wrong connector for this command.

ENOTARMED	Insulation requested without arming first.
ERANGE	Argument out of range.
EHW	Hardware not ready.
ESYNTAX	Command not understood.

6. Key Design Decisions

4-wire (Kelvin) resistance measurement

An earlier design measured resistance using two connections only, so switch resistance added error to every reading. A second, separate sense path was added that carries no current, so it adds no error — see MatrixCard_SetSensePaired() in Section 3.5.

Measuring resistance as a ratio against a reference resistor

The test current's own accuracy is not high enough on its own. Kelvin_MeasurePair() also reads a precision reference resistor using the same current and computes the result as a ratio (ADS124S08_OhmsRatiometric(), Section 3.7) — this cancels the current source's error out of the result entirely.

Insulation test method

Insulation_TestPair() applies HV to one wire while grounding the rest and reads the leakage. Relays are only switched while HV is off (HvCard_ConnectPair() is always called before HvCard_SetVoltageFraction(), never after), to protect the relay contacts.

Shared bus addressing across cards

The Matrix Card and every HV card use identical chip addresses on their shared I2C bus. Exactly one card's connection to that bus is enabled at a time — see board_init_hv() in Section 3.2.

Abort must work immediately, even mid-run

tComms is given a higher priority than tSequencer specifically so ABORT is always received and acted on during a long run — see Section 3.3 and the ABORT note in Section 3.6.

7. Known Limitations

Implemented in the current code but not yet verified against real hardware:

- The ratiometric resistance measurement (ADS124S08_OhmsRatiometric() path) has not been checked against a wire of known resistance on real hardware.
- The insulation (HV) test has not been run against real hardware through the GUI.
- Board_ScanBus() has not been verified on real hardware.
- DS18B20_ReadTemperature() has not been tested against a physical sensor.
- board_init_hv() currently supports up to three HV cards (BOARD_HV_COUNT <= 3); a fourth requires a hardware connection that has been agreed but not yet built.

Appendix A: System Reference

Peripheral and pin assignments, for an engineer adapting the firmware to a hardware revision.

Peripheral	Used for	Notes
I2C bus 1	Matrix Card switching control	Local to the Control Card only, not shared.
I2C bus 2	HV card relay control; Matrix Card resistance-ADC control	Shared. Exactly one card enabled at a time.
SPI bus 1	Resistance-measurement ADC (Matrix Card)	24-bit precision ADC (ADS124S08).
SPI bus 2	Per-HV-card: HV output DAC and two sensing ADCs	Isolated for safety.
SPI bus 3	Continuity/resistance front-end ADC (Control Card)	12-bit ADC (AD7476).
1-Wire (bit-banged GPIO)	Board temperature sensor	Diagnostic only; does not gate readiness.

Driver	Chip type	Role
mcp23017	16-bit I/O expander (I2C)	Drives relay coils and switching-chip control lines.
ads124s08	24-bit precision ADC (SPI)	Resistance measurement; also supplies the test current.
ad7476	12-bit ADC (SPI)	Continuity sensing; HV rail and leakage sensing.
dac8830	16-bit DAC (SPI)	Sets the high-voltage output level.
ds18b20	Digital temperature sensor (1-Wire)	Board temperature monitoring.

Appendix B: Glossary

Term	Meaning
Stage 1 / Stage 2	Continuity and resistance testing, harness on the Matrix connector.
Stage 3	Insulation testing, harness on the HV connector.
Force connection	A connection that carries the test current.
Sense connection	A connection that measures voltage only, carrying no current.
Arm	Explicit confirmation required before high voltage can be applied.